CELEST: Federated Learning for Globally Coordinated Threat Detection

Talha Ongun, Simona Boboila, Alina Oprea, *Member, IEEE*, Tina Eliassi-Rad, Jason Hiser, and Jack Davidson, *Life Fellow, IEEE*

***Abstract*—The cyber-threat landscape has evolved tremendously in recent years, with new threat variants emerging daily and large-scale coordinated campaigns becoming more prevalent. In this study, we propose CELEST (CollaborativE LEarning for Scalable Threat detection), a federated machine learning framework for global threat detection over HTTP, which is one of the most commonly used protocols for malware dissemination and communication. CELEST leverages federated learning in order to collaboratively train a global model across multiple clients who keep their data locally. Through a novel active learning component integrated with the federated learning technique, our system continuously discovers and learns the behavior of new, evolving, and globally-coordinated cyber threats. We show that CELEST is able to expose attacks that are largely invisible to individual organizations. For instance, in one challenging attack scenario with data exfiltration malware, the global model achieves a three-fold increase in Precision-Recall AUC compared to the local model. We also design a poisoning detection and mitigation method, DTrust, for federated learning in the collaborative threat detection domain. We deploy CELEST on two university networks and show that it is able to detect the malicious HTTP communication with high precision and low false positive rates. Furthermore, during its deployment, CELEST detected a set of 42 previously unknown malicious URLs and 20 malicious domains in one day, which were confirmed to be malicious by VirusTotal.**

***Index Terms*—Federated learning network security, intrusion detection, machine learning (ML).**

I. Introduction

MODERN cyber attacks have become sophisticated, coordinated, and operate on a global scale. We have witnessed globally-coordinated campaigns with the ability to spread to hundreds of thousands of victim machines on the Internet [1], [2]. While attackers exploit a wide range of vulnerabilities in various protocols, HTTP has become one of the prevalent communication protocols for malware dissemination [3]. To the attackers' advantage, malicious communication over the HTTP protocol can easily blend in with the large volumes of benign traffic and is rarely blocked. Existing defenses against HTTP malware include network intrusion detection systems, as well as machine learning (ML) methods applied to domain names [4], [5], URLs [6], [7] or HTTP logs [8], [9], [10], [11], [12], [13]. However, these methods are usually used within a single organizational network and have limited capability to detect attacks not seen at training time.

An important open question for thwarting global malware on the Internet is how to leverage defender collaboration and enable coordinated cyber defenses. To date, inter-organizational cooperation has been used primarily to share threat intelligence in the form of Indicators of Compromise (IoC), such as IP addresses, domain names, and URL patterns used during an attack [14], [15]. However, this approach has well-known limitations as it relies on detection of ongoing attacks and their associated IoCs, while attackers can change their infrastructure and behavior to make the detected IoCs obsolete [16], [17], [18]. This observation leads to the question: *Are there other, more proactive and reliable approaches to global defense coordination that could be effective against evolving, sophisticated cyber threats?*

In this work, we answer the above question affirmatively by presenting CELEST, a federated machine learning framework for global HTTP-based threat detection. CELEST enables collaboration among defenders to globally train neural network models for HTTP malware detection. The main goal in designing CELEST is to facilitate *knowledge transfer* between participants through a collectively trained model. A global machine learning model captures data diversity and thus becomes a powerful tool for uncovering a wider range of malware behavior characteristics. This approach enables clients to detect malware seen for the first time in their networks, which a locally trained ML model cannot identify.

In real-world deployments of ML threat detection systems, the vast majority of data is unlabeled, and ground-truth labels for malicious samples are limited. In addition, supervised defenses often fail to detect novel attacks that have not been previously encountered in training, and it is difficult to obtain accurate labels of malicious activity on a network. To address these concerns, we propose the design of a new active learning component as part of the CELEST federated learning framework with the goal of increasing the labeled

Received 9 June 2024; revised 8 June 2025 and 2 August 2025; accepted 15 September 2025. Date of publication 29 September 2025; date of current version 9 October 2025. This work was supported in part by the U.S. Army Contracting Command-Aberdeen Proving Ground (ACC-APG) under Contract W911NF-18-C0019, in part by the Defense Advanced Research Projects Agency (DARPA) under Grant W911NF-21-10322, in part by the U.S. Army Combat Capabilities Development Command Army Research Laboratory [ARL Cyber Security Cyber Research Alliance (CRA)] under Agreement W911NF-13-2-0045, and in part by NSF under Grant CNS-1717634. The associate editor coordinating the review of this article and approving it for publication was Prof. Fengwei Zhang. *(Corresponding author: Talha Ongun.)*

Talha Ongun, Simona Boboila, Alina Oprea, and Tina Eliassi-Rad are with Northeastern University, Boston, MA 02115 USA (e-mail: ongun.t@northeastern.edu).

Jason Hiser and Jack Davidson are with the University of Virginia, Charlottesville, VA 22904 USA.

Digital Object Identifier 10.1109/TIFS.2025.3614442

set during the training phase and, eventually, enhancing the global model's detection capabilities. In our design, we use active learning to identify a small set of anomalous samples for investigation in each round of training, and augment the training set with maliciously labeled samples.

We evaluate CELEST using a large dataset of HTTP logs collected at the borders of two university networks. We also use three public datasets from different malware families (Mirai, Gafgyt, and the data exfiltration malware dataset from [9]), as well as attack recreation data generated on the university networks. We show that, across all the malware families we considered, the global model outperforms local models trained on a single client's data. The improvements obtained by global models are significant. For instance, the global model is able to detect a data exfiltration malware family with three times higher Precision-Recall AUC (PR-AUC) than the local model. Importantly, global defenses enable clients to detect new malware in their environments (i.e., malware that they have not seen in training), which is learned from the models shared by other clients participating in the federated protocol. We further demonstrate that, with active learning enabled, CELEST can detect *entirely new malware for which no labels are available in the training data of any client*. In particular, the key intuition to enable new malware discovery is to include an anomaly detection module trained on each client network with the goal of detecting and labeling anomalies in the network. Often, new attack instances will result in anomalous traffic relative to the benign traffic. Once the anomalies are confirmed as attack instances, they are added to the next round of federated learning training, and the global model will learn to recognize these newly discovered attacks. One of the most important threats against federated learning is adversarial manipulation by malicious or compromised clients to poison the model [19], [20], [21], [22], [23]. We design a new defense technique specific to threat detection called DTrust (Distributed Trust), with the goal of training in the presence of poisoning attacks by identifying and removing the malicious clients. The benign clients evaluate locally whether the global model received from the server can be trusted, and notify the server if a large performance degradation is observed. The server investigation consists of inspecting individual model updates and identifying the clients that sent "bad" updates to remediate the attack. DTrust relies on the insight that most client organizations in a collaborative threat detection system act in good faith, and that they are incentivized to actively participate in defense and validate the model during the training process to protect against poisoning attacks. We evaluate DTrust in three poisoning scenarios in which a number of compromised clients inject different attack patterns to be misclassified by the global model. In one scenario, the global model's PR-AUC degrades from 0.93 to 0.11 when no defense is deployed, but DTrust restores the PR-AUC to 0.93 after identifying the malicious clients and removing them from training.

Finally, we deploy CELEST on two university networks using three attack recreation exercises performed at intervals of several months. We show that CELEST detects the malicious communication carried out during the attack recreation with high precision, and with false positive rates lower than 3.3×10^{-5} on both networks. Moreover, the model trained in the first experiment maintained strong performance during an evasive attack exercise conducted four months later. In addition, CELEST detected a set of 42 previously unknown malicious URLs and 20 domains on the two networks, which were confirmed malicious by VirusTotal. To summarize, our contributions are:

- **Federated learning for global cyber defense:** We design CELEST, a scalable and privacy-preserving framework for federated training of global neural network models, which enables early-stage HTTP malware detection at participating organizations.
- **Active learning for limited ground truth:** We introduce a novel active learning component in our federated design, which selects samples for investigation and labeling using an anomaly detection module, and thus enables the discovery of completely new attacks. To the best of our knowledge, this is the first exploration of active learning within the federated threat detection domain. It combines rare-class discovery with the ranking of the global classifier, enabling the detection and learning of attacks that were not observed locally.
- **Poisoning mitigation:** We design DTrust, a new poisoning detection and mitigation method specifically designed for federated learning in the collaborative threat detection domain. The main innovation of DTrust is having the clients bootstrap (publicly verifiable) trust; this is in contrast to previous approaches that rely solely on the server to detect poisoning attacks.
- **Comprehensive evaluation:** We evaluate CELEST using large-scale datasets from two university networks, public traces from three malware families, and three attack recreation exercises. We show that global models trained in CELEST have high precision and recall, and low false positive rates, improving significantly upon local models. We further demonstrate the impact of poisoning attacks and effective mitigation with DTrust, which outperforms a poisoning defense based on weight clipping.
- **Deployment and detection of unknown malware:** We deploy CELEST on two university networks and find instances of previously unknown malware (42 malicious URLs and 20 malicious domains).

II. Problem Statement and Threat Model

In this section, we discuss the problem of HTTP malware detection along with the limitations of existing solutions, and introduce our system requirements and threat model.

HTTP-based malware. HTTP is one of the protocols most widely used by adversaries to perpetrate malicious activities, such as data exfiltration [24], vulnerability exploitation [25], and covert channel communication [26]. Command-and-control (C2) servers often use HTTP to send commands to compromised systems and receive stolen data [27]. Prior work proposed a variety of methods for detecting malicious network activity over HTTP, including URL-based detection [7], [28], detection using web-proxy logs [8], [10], [11], [12], [13], and application fingerprinting [9], [29], [30]. We focus

on malicious activity detection using HTTP logs for several reasons. First, while malicious URL detection has been shown to be successful in mitigating certain attacks (e.g., phishing), HTTP-based detection methods are able to address a variety of attack surfaces used by different malware families. Second, the HTTP-based methods avoid deep inspection of the payload which could be obfuscated by attackers, and rely only on the HTTP headers for increased scalability and lower computation costs. Lastly, these approaches utilize common logs collected by web proxies installed at the border of enterprise networks, which most enterprises use as part of their perimeter control.

Most of the existing machine learning detection techniques on HTTP logs attempt to detect malware activity within a single network by training local models on labeled data available in that network [8], [10], [12], [13]. As attackers employ various techniques to evade detection, such as changing the C2 protocol, or changing the domain names and the IP addresses of the C2 infrastructures, local models will fail at detecting these. In this work, we address the problem of *designing global detection of HTTP malware through coordination among multiple participating organizations*. We believe that collaboration among multiple defenders is critical to enable a better and more resilient cyber defense strategy and will lead to more effective threat detection methods. Unfortunately, the state of the art in collaboration among multiple defenders is based on threat intelligence sharing platforms, which enable sharing of indicators of compromise (IoCs) after attacks are detected. IoCs have limited utility at detecting cyber threats, particularly as IoCs become stale with small changes to attackers' malicious infrastructures or communication protocols. We are interested in more proactive, coordinated approaches among defenders that can resist evolving cyber attacks.

In designing our system, we identified several requirements for real-world deployment in participating organizations:

- **Globally coordinated detection**: We are interested in global methods that expose attacks promptly, when they are still largely invisible to individual organizations.
- **Real-time processing**: We require real-time processing of HTTP logs to generate predictions on individual HTTP logs when the model is deployed. This enables early-stage attack detection and thus minimizes the damage.
- **Scalability**: We would need to run the system on multiple networks with large volumes of logs.
- **Log privacy**: The system should require minimal data sharing across participating clients, in order to ensure the privacy of sensitive security logs.
- **High accuracy**: To be used in production, the system should have high accuracy, precision, and recall, as well as low false positive rates across different classes of HTTP-based malware attacks.

Challenges. Detecting malicious communication over the HTTP protocol is challenging for multiple reasons: First, the malicious traffic transmitted on the HTTP protocol blends in with large volumes of legitimate traffic generated by users, making detection very difficult. Second, there is a high imbalance between malicious and benign samples on a single network, making it challenging to train supervised learning methods with high accuracy and low false positive rates [31]. Third, most existing systems for HTTP malware detection employ some form of aggregation of events from multiple logs. For instance, beaconing detection methods apply time series analysis on multiple communication events to the C2 server to detect periodic communication [10]. It is challenging to detect malicious HTTP communication in real time, as it requires generating a prediction on individual log events.

Threat Model. We aim to design a globally coordinated system with multiple participating clients for detection of malicious activity over the HTTP protocol. The malicious communication could be part of multiple stages of a malware campaign, including malware delivery, the C2 communication with the malicious server, and data exfiltration activities. We assume that the attacker compromises one or several victim machines within the participating networks. Hosts in the network can be infected in a variety of ways, such as vulnerability exploits, social engineering, and drive-by download attacks. The infection vector might not occur over HTTP, and therefore our system might not be able to detect the initial infection. However, our goal is to detect any subsequent malicious communication over HTTP, which is a common communication channel once attackers have established a foothold in a network.

Our system analyzes HTTP logs collected at the borders of the monitored networks. The participating organizational networks run local computation on the collected logs for training local models, and share those with a central server that aggregates a global model. We assume that the central aggregator correctly follows the protocol for aggregating the local updates into a global model. However, the clients participating in the protocol might be subject to data poisoning attacks (in which the logs they collect are under the control of an adversary), or model poisoning attacks (in which the adversary controls the updates sent to the central aggregator). Our goal is to train models that identify the malicious activity over HTTP, while providing resilience against poisoning.

III. System Overview and Methodology

In this section, we present an overview of CELEST, followed by a detailed description of each system component. CELEST is a federated framework for cyber defense, where a set of participating clients collaboratively learn a global model G from HTTP logs collected at their network border. The goal of the global model is to learn a classifier that generates predictions regarding whether individual HTTP logs are Malicious or Benign. We consider the *cross-silo federated learning* setting, suitable for a relatively small number of clients, in which all clients participate in each round of training [32]. Following the federated learning paradigm [33], in a training iteration $t \in \{1, \ldots, T\}$, each client $i \in \{1, \ldots, n\}$ locally trains a local model W_i^t based on the previous global model G_{t-1}, by performing stochastic gradient descent (SGD) updates on a subset of its local data, D_i^t. The clients send their local model updates to the server, which aggregates them to produce the updated global model G_t and distribute it to the clients. The process continues iteratively until the global model converges.

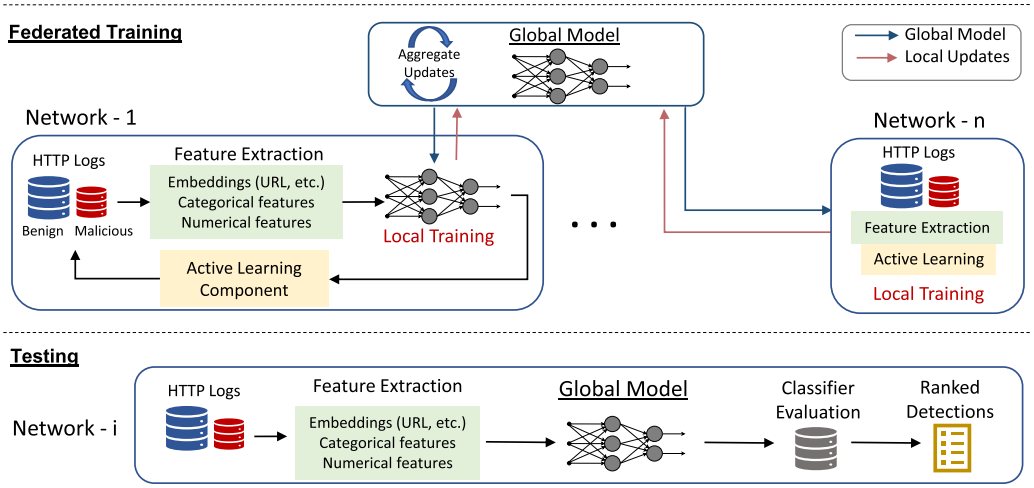

Fig. 1. Overview of the CELEST federated learning framework. The system is designed to train a global model for HTTP malware detection where multiple networks participate as clients in the FL protocol. The system uses an embedding model for feature extraction, and local model updates are aggregated at a central server to build a global model that is capable of detecting malicious behavior across the networks. New attack discovery using an active learning component is integrated to augment the labeled instances through anomaly-based sample selection.

The training data (HTTP logs) D_i maintained by each client i locally is labeled as malicious or benign. We discuss in Section IV how we generate ground truth for data labeling. Before training, the data undergoes a feature extraction phase where individual HTTP logs are processed to generate a set of 5862 features across three categories – embedded, numerical and categorical (Section III-A). We propose the use of embedded features from text-based HTTP fields (URL, domain and web referer), adapting word embedding representations from natural language processing (NLP). Word embeddings are generated with deep learning models such as Word2Vec [34] and FastText [35], which are trained to capture contextual and semantic similarities in the text, ensuring that similar words are positioned closer in the embedding vector space.

CELEST uses federated learning techniques for two tasks: (1) generating embedded feature representations, and (2) training a neural network classifier for detection of malicious HTTP traffic. For the first task, we introduce a federated method in which each client updates sequentially a shared embedding model using their own data corpus. For the second task, clients use their local HTTP feature vectors to collectively train a global model that is able to detect various attack patterns. CELEST introduces a novel active learning component that uses a local anomaly detection module to augment the ground truth of malicious activities through sample selection (Section III-B). In addition, CELEST employs a novel distributed defense mechanism against poisoning attacks, in which the clients themselves help identify malicious attempts to corrupt the global model (Section III-C).

We emphasize the challenge of designing threat detection systems that achieve high accuracy while maintaining low false positive rates. To this end, our framework leverages multiple inter-connected machine learning components: (1) an unsupervised federated embedding model for URL representation; (2) a supervised federated learning model for global threat detection; (3) an active learning component that uses an unsupervised anomaly detector for sample selection in order to generate additional ground truth for the global supervised model.

Figure 1 illustrates the overview of the system. We detail each component of CELEST in the following sections.

A. *Feature Representation*

Our system is unique in its requirement for real-time processing of HTTP log events. Thus, timing features and aggregated features over multiple HTTP logs or flows are not applicable to our setting. We leverage all the available fields in HTTP logs, except the source IP address (since our detection methods are not specific to the host machine) and source port (which does not carry information related to our detection task). We include three feature categories: (1) *embedding features* for domain, URL, and web referer representation, (2) *categorical features* for external IP subnet, port, user agent string, method, status code, and content type, and (3) *numerical features* for request and response size, transaction depth, and browser version.

URLs are one of the HTTP fields particularly susceptible to adversarial manipulation; they are often used by malware to communicate information with C2 servers through various parts of the URLs, including sub-domain, parameters, query string, file name, and fragment. Hence, it is important to have an effective, semantics-aware feature representation of URLs. Unlike prior work on URL feature representation [6], [7], [36], [37], [38], [39], we propose an embedding model that preserves the semantic structure of the URL, handles new tokens at deployment time, and can be trained in a federated fashion.

Embedding model design. In Figure 2, we present our design for creating the embedding representation in a federated unsupervised manner. Initially, we parse the URLs into tokens representing different categories (e.g., domain, path, query string) to preserve the structure of a URL. Each token is considered a word, and each URL is viewed as a sentence of words.

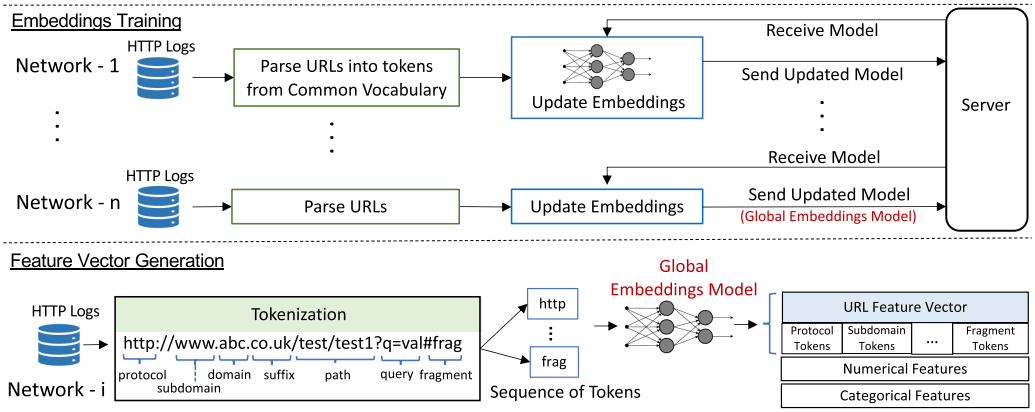

Fig. 2. Federated embedding model training for URL representation. We generate embedded features for URL, Domain and Referer. In addition to embedded features, we also include numerical features and categorical features.

We considered several approaches for generating URL embeddings. The first option is a centralized training of the word models using either Word2Vec or FastText architectures, where the training of the embedding model is carried out on the server using the data collected from all the clients. Thus, a single entity (i.e., the server) has access to all the data, and uses it to learn the vector embeddings from the URLs (or domain, referer). This approach suffers from scalability and privacy issues, as all the raw URL data has to be collected in a central server to perform the training. We are interested in a more privacy-preserving and scalable approach for distributed training of the embeddings among multiple participants. It is possible to apply the Federated Averaging algorithm to train Word2Vec or FastText models. However, in practice, Federated Averaging for Word2Vec has been shown to exhibit slow convergence due to the large size of updates sent in each iteration [40]. Based on these considerations, we developed a distributed approach using integrated sequential client updates. In this approach, each client has access to its own data and updates the global embedding model locally, using its entire corpus. The client sends the updated global model back to the server, which acts as a trusted central coordinator. The clients apply their updates sequentially, in a round robin fashion, over multiple iterations. This method still requires a common word (i.e., token) vocabulary. To address this, clients send their token frequencies to the trusted server in a pre-processing phase. The server aggregates these frequencies, determines a lower bound (e.g., all tokens appearing more than once), and returns the resulting global vocabulary to each client.

To avoid the need for vocabulary sharing, FastText embeddings can be used instead of Word2Vec. FastText represents tokens as character n-grams (i.e., sequences of n characters), which can be constructed independently by any participant, including the server, from publicly available datasets to preserve privacy. In this approach, the set of n-grams is derived deterministically from a predefined public corpus, and both the server and clients apply the same preprocessing procedure to generate an identical n-gram vocabulary and aligned embedding indices prior to training. Alternatively, the central server can generate the vocabulary and indices from the corpus and distribute them to clients to ensure consistency. Once these dictionaries are constructed and indexed consistently, embedding training can proceed in a distributed manner. This design is practical in domains such as URL-based tasks, where common substrings are prevalent, and it eliminates the need for clients to share local word dictionaries.

FastText has the additional advantage of supporting new tokens not observed at training time by generating n-gram representations for them. This property is important because adversaries can change parts of the URLs to evade detection (e.g., the query string, path, and parameters can be easily updated). As we show in our evaluation, FastText and Word2Vec achieve similar performance (and have higher performance than lexical features, as expected). For these reasons, we select federated FastText as the preferred embedding method for URL representation.

B. Active Federated Learning

In this work, we employ the commonly used Federated Averaging (FedAvg) [33] training method, where the client models are weighted by their dataset size and averaged to update the global model as follows: $G_t = \sum_{i=1}^{n} \frac{\|D_i^t\|}{\|D^t\|} \times W_i^t$, where $\|D^t\| = \sum_{i=1}^{n} \|D_i^t\|$. The local training on each client is carried out using a Feed-Forward Neural Network (FFNN) model based on our feature representation.

Training supervised models such as FFNN requires labeled data, a known challenge in cyber security [31] where the vast majority of data available in real-world deployments is unlabeled. Furthermore, existing defenses that might work well on previously seen attacks often fail at detecting new emerging threats, making it particularly challenging to get accurate labels of malicious activity on a network. To address these concerns, we integrate an active learning component into CELEST with the goal of augmenting the ground truth of malicious activities incrementally at each iteration of the training phase. Thus, the global model's detection capabilities improve in a streaming fashion, similar to online learning where true class labels of new incoming samples are usually unknown [41]. This approach fits particularly well with federated learning, which is designed for continuous training over time, and thus can account for malware evolution over time. To the best of our

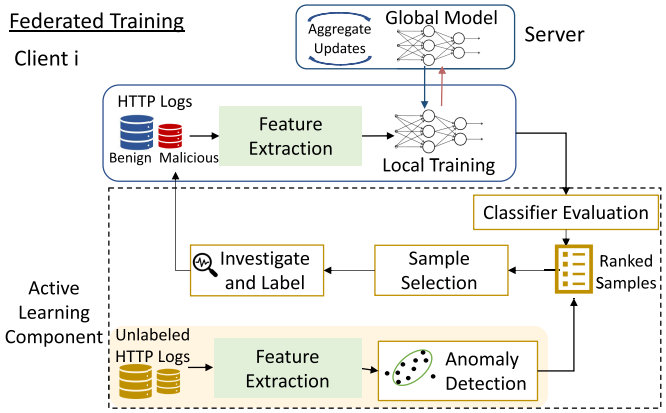

Fig. 3. The Active Federated Learning framework. At time step t, the global model G_t is used to rank the unlabeled data of client i. In addition, an anomaly detector is used to rank the remaining unlabeled data samples by their anomaly score. The top-ranked samples are investigated and labeled by a human expert and used in the next iterations.

knowledge, CELEST is the first threat detection system using active learning in a federated method of training with limited labeled data. Several strategies for selecting instances to be labeled in active learning have been previously explored, such as random sampling, uncertainty sampling [42], expected error reduction [43], and variance reduction [44]. In our case, we are interested in expanding the malicious ground truth, therefore we combine both uncertainty sampling and anomaly detection in a hybrid approach for sampling the most suspicious HTTP log events to be investigated, as proposed by prior work [45].

Figure 3 illustrates the Active Federated Learning mechanism. At time step t, the global model G_t is used to rank the unlabeled data of client i. In addition, an anomaly detector is used to rank the remaining unlabeled data samples by their anomaly score. The top-ranked samples are investigated and labeled by a security analyst and used in the next iterations. In real deployments, the labeling can be partially automated using threat intelligence to reduce the workload on human experts. We consider a small budget b of samples that are investigated and labeled by a human expert at one of the participating organizations i, and time step t. The first strategy consists of using the global model G_t, represented by a Feed-Forward Neural Network classifier, to evaluate and rank the unlabeled dataset; the most suspicious $b/2$ samples are then investigated and labeled by a security analyst. These are likely similar to previously seen malicious samples. The second strategy consists of using a local anomaly detection module to rank the remaining unlabeled data according to an anomaly scoring method; the most anomalous $b/2$ samples are then investigated and labeled by a security analyst. The second strategy identifies anomalous logs that could augment the ground truth with new attack samples not seen in training before. The combination of the two strategies is critical for CELEST's ability to detect attacks with very limited ground truth. The samples labeled malicious are added back to the training dataset of client i and used in the next iterations of federated learning. We focus on labeling malicious samples only, as some adversarial actions might look benign during stealthy attacks. We assume the human analysts make the correct labeling decision, and note that learning from noisy labels is an extensively studied ML topic on its own, not specific to our system [46], [47].

We demonstrate in our evaluation section that active federated learning can be very powerful in some scenarios, even when starting the FL training process with zero labels. Our anomaly detection module uses the well-known Isolation Forest algorithm [48]; however other anomaly detection methods such as local outlier factor (LOF) [49], one-class support vector machines (SVM) [50], and clustering [51] can also be employed. To increase resilience, we train an ensemble of Isolation Forest models on multiple time windows, where each time window corresponds to one FL iteration (time step). Specifically, given a client i, the anomaly detector is trained on unlabeled data from the previous k time windows (i.e., the datasets $D_i^{t-k}, \ldots D_i^{t-1}$), while anomaly detection is carried out on the current time window t (i.e., the dataset D_i^t). For instance, a good trade-off between speed and performance was reached in our experiments at $k = 3$.

C. Resiliency to Poisoning Attacks

Due to its distributed nature, federated learning is vulnerable to adversarial manipulation by malicious or compromised clients. Poisoning attacks that corrupt the training data [52] or the client model updates [19], [20], [21], [22], [23] have been an important attack vector studied in recent years. CELEST might be vulnerable to either data poisoning attacks, in which the logs collected by compromised clients are under adversarial control, or model poisoning attacks, in which malicious clients might send arbitrary model updates to the central aggregator.

Existing defenses against poisoning attacks in federated learning can be classified as: (i) defenses that limit the contribution of each client to the global model [20] or perform anomaly detection to remove specific client updates [53], [54], [55]; (ii) defenses against backdoor poisoning that detect and mitigate the presence of a trigger backdoor [56], [57]; (iii) defenses that leverage a trusted dataset at the server to filter malicious updates [58]. We are interested in developing a general defense against both data and model poisoning attacks that leverages the specifics of our threat detection setting.

1) DTrust Design: We propose a novel defense algorithm called DTrust (Distributed Trust), with the goal of training a resilient federated model in the presence of poisoning attacks. We introduce a novel distributed scheme of bootstrapping trust, in which the clients take an active role in detecting poisoning attacks. This approach is particularly suited for our threat model, where the clients are organizations (rather than IoT devices or individual machines), and therefore have enough resources to actively participate in defense. DTrust fully exploits the distributed nature of FL, in contrast to previous trust-based schemes that rely on the server to detect adversarial attempts, such as ERR, LFR [21], and FLTrust [58].

Each client i interested in participating in CELEST's DTrust defense maintains a small dataset D_i^{Trust} called the *trust dataset*. Multiple clients use their locally available data D_i^{Trust},

which includes locally collected and verified malicious and benign samples covering a variety of attack behavior. Each client tracks the global model performance on this dataset, and shares it with the server upon detecting performance degradation. The server leverages publicly available tools (e.g., VirusTotal, third-party threat intelligence platforms, etc.) to ensure that the trust datasets are clean. For instance, the trust dataset could contain command-and-control traffic to a domain that has a high score on VirusTotal. Such tools may not be readily available in other application domains (e.g., image classification, human activity recognition, etc.) [58], making this approach particularly feasible for our cyber security setting.

DTrust addresses major challenges present in our setting through a series of design choices. We next describe potential challenges and key insights that ensure DTrust's correct and efficient operation:

- Malicious clients reporting false information: To counter false reporting, the server ensures that the trust dataset is "clean", i.e., verifiable via publicly available tools.
- Attacker gradually shifting the model: The server maintains a history of intermediary global models and compares the potentially malicious model against the history. The server is thus able to detect repeated model shifts that cumulatively push the performance degradation, and roll back to earlier versions. This approach increases DTrust's resilience to stealthy poisoning attacks that happen across multiple training time steps.
- Performance impact: The impact is minimal, as the server maintains a history of intermediary models, and does not re-train upon detection.

2) DTrust Algorithm: When a client receives the global model G_t from the server, it evaluates the model on its trust dataset. In our implementation, we use the cross-entropy loss metric L, although other performance metrics could also be used (e.g., error rate, PR-AUC score, F1-score). The *loss impact* at client i is defined as $\frac{L-L_{min}}{L_{min}}$, where L, L_{min} are the current loss and the minimum loss across previous iterations, respectively. If the loss impact exceeds a threshold T, then the client notifies the server and sends: (1) the iteration number t_{best} of the previous best-performing global model, and (2) its trust dataset D_i^{Trust}.

The server maintains a history database H, containing: (1) the previous global models $G_1, \ldots, G_{t-1}$, and (2) the most recent k model updates $W_i^{t-k}, \cdots, W_i^{t-1}$ received from each client i. When the server is notified of a large performance degradation by client i, it uses i's trust dataset to investigate whose client updates have caused a significant performance drop, as follows:

1) Confirm that the trust dataset D_i^{Trust} sent by client i is legitimate, i.e., that it contains malicious samples that can be verified using threat intelligence tools.
2) Validate that the global model at t_{best}, i.e., $G_{t_{best}}$, performs well on the trust dataset, whereas the current model is significantly worse.
3) Run the following loop to investigate each client m:
   a) Leave one client m out and aggregate a global model G' without m's latest updates after t_{best}.
   b) Evaluate the global model G' on the trust dataset.
   c) Compute the loss impact caused by client m as $\frac{L-L'}{L'}$, where L, L' represent the loss of the global model with and without client m, respectively.
   d) If the loss impact exceeds a certain threshold, conclude that client m may be poisoned, remove it from the set of clients, and revert the model to exclude its updates. Communicate the observation with client m to investigate the incident.

IV. Evaluation

Datasets. We evaluate our system on several data sources. Each client organization participating in CELEST maintains a labeled set of malicious and benign samples. We use traffic from the two university networks as benign data and employ various malicious data sources for different experiments, as explained below. Similar to previous research using data reduction [10], [12], we filter the datasets based on domain popularity using Tranco [59] to exclude top 10,000 domains. This filtering is not expected to affect the detection results, since the malicious connections from the malware datasets had no matches in Tranco's most popular domains.

(A) University Network Dataset: We obtained access to HTTP logs from two large university networks. The networks contain an average of 20 and 9 million HTTP log events per day, respectively, and were used actively during the data collection period. Sensitive fields in the logs (such as internal IP addresses and URL parameter values) are anonymized in a consistent manner to protect users' personal information. We performed all our analysis on servers within the university network. The IRB office at one of the universities reviewed our data collection process and determined that this research does not qualify as Human Subject Research. However, our team members participated in IRB training and used best practices to handle network data. We remove the most popular Tranco [59] domains and external domains contacted more than 10,000 times from this dataset. Both sets of domains are unlikely to serve malware, and a similar methodology was used in previous work for data reduction [10], [12]. After filtering the traffic, we have between 1,900 and 31,000 logs per 30-minute time interval.

We used several methods to ensure that the university datasets are largely benign. Both institutions had operational Security Operations Centers (SOCs), network IDS and monitoring tools in place at the time of data collection to detect and respond to threats. In collaboration with their IT teams, we reviewed security alerts and excluded any traffic associated with known incidents from the dataset. We also queried VirusTotal for samples identified with anomaly detection during experimentation to remove known malicious samples from the dataset. We acknowledge the possibility that some undetected malicious activity may remain in the unlabeled data. However, this reflects the realities of operational environments, where datasets are rarely perfectly clean. Our evaluation focuses on assessing detection performance across specific attack classes, which remains valid even under these conditions.

(B) Network IDS logs (NIDS) In one of the university networks, a commercial off-the-shelf network IDS is available, and alerts for potentially malicious events are collected in real time. We have access to the historical samples, and we collect malicious domains and IP addresses from the malware-callback alerts. We process 28,985 alerts, containing 9,124 unique requests and 687 unique hosts (domain and IP addresses). We use the collected indicators to match and label connections in each network as malicious during the training period.

(C) IoT Malware Dataset (Mirai, Gafgyt) We collect a dataset of IoT malware (Mirai and Gafgyt) from CyberIoCs, VirusTotal, VirusShare, and malicious samples shared by Alrawi et al. [60]. We build a dynamic analysis framework to execute the malware in a sandboxed environment in order to collect the HTTP logs for each malware family. In total, we use 300 malware samples from each family, containing 20,627 and 13,889 HTTP events.

(D) Data Exfiltration Malware Dataset (DEM) Bortolameotti et al. [9] collected HTTP logs for data exfiltration malware samples in a sandbox and released the dataset. This dataset contains multiple malware families: Shakti, FareIT, CosmicDuke, Ursnif, Pony, Spyware, and SpyEye, and has more variability. We use a total of 79 malware samples containing 8,843 HTTP events from this dataset. Over the course of several experiments, we merge the traffic of IoT and DEM malware at training and testing time to create controlled experiments with known ground truth of malicious activities.

Implementation. We implemented the system in Python 3.8, and used *Keras [61]* and *TensorFlow Federated [62]* for federated training. We use the *Gensim* framework [63] for training embedding models.

Hyper-parameters. The neural network hyper-parameters are selected after a grid search. We use a single hidden layer of size 128, and a dropout layer with rate 0.1 and learning rate of 0.01. Federated learning updates happen within rounds, and we set the duration of a round to $t = 30$ minutes worth of data. At this frequency, we expect less impact from the communication overhead. In real-world deployments, communication costs can vary significantly depending on factors such as network infrastructure, client hardware, and bandwidth availability. We recognize the importance of this trade-off. In future work, a real-world deployment scenario could be used to assess the impact of varying the communication frequency on both performance and efficiency.

Evaluation Metrics. We evaluate our system using Precision and Recall metrics, as our dataset is also imbalanced. We measure the performance with Precision-Recall Area Under Curve (PR-AUC), a single metric that captures the performance across all thresholds. We also use False Positive Rate (FPR) as an important metric for the models to be deployed in a real-world setting.

Feature Set Investigation. We compare different URL representation methods: embeddings-based features constructed in federated and centralized settings, as well as lexical features [7]. Federated embedding models provide similar performance to the centralized models, and the lexical feature representation performs worse on our data, with 10% lower recall compared to the federated FastText method. This result implies that the embedding method generalizes better at detecting a larger number of malware samples, which can evade the lexical features. We further demonstrate the benefit of additional features (e.g., the IP address, referer and user agent) extracted from HTTP logs to augment the URL embeddings.

A. Federated Models Vs Local and Centralized Models

In this section, we investigate how a network can benefit from federated training when a new malware has been previously detected in one of the collaborating networks, but is seen locally for the first time. We design experiments to use different malware families in the two networks during training, and show how the global model helps with detection of a malware family seen for the first time in one of the networks. In particular, we generate a set of controlled experiments with the two university networks as clients. We merge malware traces from the three families (Mirai, Gafgyt, and DEM) in both networks to determine the performance of the local and global models under multiple scenarios. We split each malware family into 80% samples for training and 20% for testing, and distribute them to the networks across rounds.

1) Knowledge Transfer Performance: Table I shows the PR-AUC of the global models trained with federated learning and the local model in various settings. The results show the improvements achieved by the global model, which successfully transfers the detection from one network to other participating clients. For example, when Mirai and DEM malware families are used for training at UNI-1 and UNI-2, respectively, the federated model achieves a PR-AUC of 0.75 at detecting DEM in UNI-1, a significant improvement over the local model's 0.26 PR-AUC. Similarly, at UNI-2, the global model achieves a PR-AUC of 0.87, compared to the local model's PR-AUC of 0.54 at detecting Mirai. In other cases, malware families share some common characteristics, which aids with local detection. In the case of Mirai and Gafgyt, a local model that trains on Mirai and later attempts to detect Gafgyt (or the other way around) performs relatively well (at 0.87-0.9 PR-AUC). Nonetheless, the global model outperforms it consistently, achieving a PR-AUC of about 0.93-0.95. We also present the false positive rate, an important metric in any detection system in a high-volume domain such as network traffic, where investigating a high number of alerts is costly. With a recall of 0.9, the global model maintains consistently low false positive rates, ranging from 0.002 to 0.01.

2) Learning Progress Over Rounds of Training: Figure 4 shows the progress during federated training for two of the experiments. As more training data is consumed, the PR-AUC steadily improves and the global model outperforms the local model in less than 10 iterations (5 hours worth of data), illustrating a clear performance gain with the federated model. This demonstrates a clear advantage in leveraging federated learning despite non-IID client data. Network traffic distributions also fluctuate on each client over time; however, the model continually improves until convergence.

3) Comparison With Centralized Training: In addition, we performed experiments to compare the federated model's

TABLE I

COMPARISON OF THE FEDERATED MODEL AND THE LOCALLY TRAINED MODELS FOR DETECTING MALWARE FAMILIES IN MULTIPLE SCENARIOS WITH DIFFERENT TRAINING SETS. EACH ROW SHOWS THE TRAINING MALWARE FAMILY IN EACH NETWORK, AND THE PR-AUC FOR DETECTING THE MALWARE THAT IS NOT LOCALLY SEEN DURING TRAINING. THE RESULTS SHOW THE PR-AUC IMPROVEMENTS OF THE GLOBAL MODELS COMPARED TO LOCALLY TRAINED MODELS AND THE LOW FPR OF THE GLOBAL MODELS

Training Malware		Deployed Network: UNI-1				Deployed Network: UNI-2			
UNI-1	UNI-2	Test	Local PR-AUC	Global PR-AUC	Global FPR at Recall 0.9	Test	Local PR-AUC	Global PR-AUC	Global FPR at Recall 0.9
Mirai	Gafgyt	Gafgyt	0.8751	0.934	0.002	Mirai	0.8806	0.9591	0.002
Mirai	DEM	DEM	0.2647	0.7522	0.01	Mirai	0.5483	0.8774	0.002
Gafgyt	Mirai	Mirai	0.8952	0.9308	0.003	Gafgyt	0.9055	0.9568	0.001
Gafgyt	DEM	DEM	0.2429	0.7548	0.01	Gafgyt	0.4464	0.8016	0.002
DEM	Mirai	Mirai	0.7231	0.8508	0.003	DEM	0.5483	0.7958	0.009
DEM	Gafgyt	Gafgyt	0.6013	0.7983	0.005	DEM	0.4736	0.7939	0.009

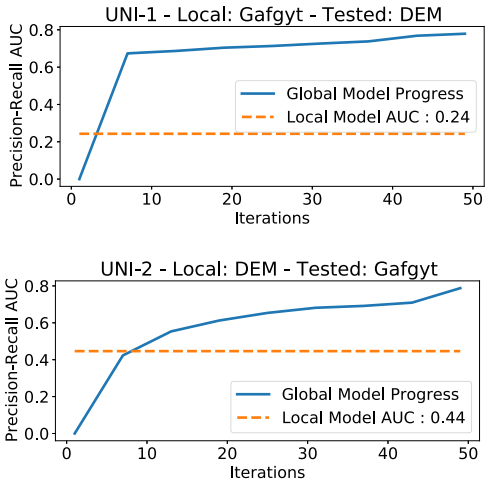

Fig. 4. Federated model progress for detecting malware families over multiple iterations evaluated on the two networks. The two sites are seeing a new malware for the first time (DEM at UNI-1 and Gafgyt at UNI-2) and attempt to detect it with their own local model and with the federated model. Each iteration processes 30 minutes of data, and the global model exceeds the local model's performance in less than 10 iterations.

TABLE II

DETECTION PERFORMANCE COMPARISON BETWEEN THE CENTRALIZED MODEL TRAINED ON AGGREGATED DATA FROM BOTH UNIVERSITY NETWORKS AND THE FEDERATED MODEL. THE FEDERATED MODEL ACHIEVES PERFORMANCE CLOSE TO THE CENTRALIZED BASELINE. BOTH MODELS SHOW SIGNIFICANT IMPROVEMENT OVER THE LOCALLY TRAINED MODEL THAT RELIES ON DATA FROM A SINGLE NETWORK (UNI-2)

			Local	Global		Centralized	
UNI-1	UNI-2	Test	AUC	AUC	F1	AUC	F1
Mirai	Gafgyt	Mirai	0.880	0.959	0.879	0.970	0.931
Mirai	DEM	Mirai	0.548	0.877	0.758	0.962	0.899
Gafgyt	Mirai	Gafgyt	0.905	0.956	0.850	0.971	0.873
Gafgyt	DEM	Gafgyt	0.446	0.801	0.824	0.899	0.861
DEM	Mirai	DEM	0.548	0.795	0.594	0.802	0.734
DEM	Gafgyt	DEM	0.473	0.793	0.602	0.814	0.767

accuracy to a *centralized model* that used the training data from both networks. We observed similar performance between the federated model and the centralized model in multiple scenarios, with a slight edge for the centralized model, as expected. For instance, in one scenario, the federated model's PR-AUC was 0.95, while the centralized model had 0.97 PR-AUC. Table II shows the detection performance comparison between the centralized model trained on aggregated data from the client networks and the federated model. Both models improve strongly on the locally trained model relying on a single network. We also include the maximum F1 score in our evaluation, to reflect the best possible trade-off between precision and recall. We acknowledge that the choice of threshold used to calculate the F1 score may vary depending on the investigation resources available to each organization and the desired balance between precision and recall.

B. Active Federated Learning

We showed how federated learning can transfer knowledge about malware observed in some networks to detect attacks seen for the first time in other networks. However, the global model will not detect a completely new attack that has not been observed in either of the participating clients in training. To overcome this challenge, we propose extending the federated learning framework with an active learning component that integrates a local anomaly detection module (Section III-B).

We design a challenging scenario to test the detection capabilities of active federated learning. The two university networks start training the global model using NIDS alerts known to be malicious, but without using the malicious labels to simulate a situation when clients are infected with unknown malware. Thus, we merge the *unlabeled* malware logs from the three malware families (Mirai, Gafgyt, and DEM) into the client networks. We test whether the anomaly detection module identifies some of the malicious samples in the local networks, and, furthermore, if the federated model with active learning is able to detect these unknown malware attacks when no labeled data from these families is initially available in training. For the anomaly detection module, we train Isolation Forest models on HTTP logs on k previous time windows of 30 minutes each, and use an ensemble of these models to detect anomalies within the current time window. We set $k = 3$, after experimenting with multiple values of k. We vary the budget used in investigation between 0 (which results in the federated model without active learning) and 500.

Table III shows the PR-AUC of detecting each of the three malware families as a function of the investigation budget. As expected, for a budget of 0, the global model performs

TABLE III

ACTIVE FEDERATED LEARNING AT DIFFERENT BUDGETS FROM 0 TO 500 IN A CHALLENGING SCENARIO WHERE NONE OF THE MALWARE FAMILIES IS LABELED IN TRAINING. WE SHOW THE PR-AUC FOR DETECTING EACH MALWARE FAMILY AT EACH UNIVERSITY NETWORK. THE FEDERATED MODEL WITHOUT ACTIVE LEARNING (BUDGET 0) CANNOT IDENTIFY NEW MALWARE (TOP ROW). THE ACTIVE LEARNING COMPONENT ACHIEVES HIGHER PR-AUC AT LARGER BUDGETS, AND GETS CLOSE TO THE MODEL TRAINED WITH ALL LABELED DATA (LAST ROW)

Active Learning Budget	Network: UNI-1			Network: UNI-2		
	Mirai	Gafgyt	DEM	Mirai	Gafgyt	DEM
0	0.19	0.13	0.21	0.18	0.06	0.22
10	0.48	0.38	0.59	0.40	0.28	0.38
100	0.70	0.60	0.66	0.63	0.49	0.52
200	0.79	0.73	0.69	0.76	0.69	0.59
300	0.83	0.76	0.66	0.81	0.73	0.61
400	0.84	0.77	0.68	0.83	0.75	0.68
500	0.87	0.81	0.70	0.84	0.76	0.69
Fully Labeled	0.97	0.96	0.82	0.96	0.93	0.72

poorly at detecting these completely new malware families (e.g., PR-AUC 0.19 at detecting Mirai at UNI-1) without active learning. Active learning used in federated training significantly helps detect new malware; as we increase the budget for investigation, the PR-AUC also increases, and with a budget of 500 samples it reaches a PR-AUC of 0.87 in the same experiment for detecting Mirai.

We also trained the active learning system with sample selection using only the anomaly detection module (without the classifier). This setup identified fewer samples per round (since it did not incorporate any samples selected by the classifier), and the detection performance was slightly worse. For instance, using both anomaly detection and the highly ranked samples generated by the classifier resulted in an increase of 11% PR-AUC compared to using only the anomaly detection module for detecting DEM. This result shows that both components for sample selection contribute to the success of active federated learning.

C. Resiliency Against Poisoning

In this section, we evaluate CELEST's resiliency against poisoning attacks, where a number of malicious clients try to impact the global model to evade detection of a specific malware pattern. In our threat model, clients belong to one of the following three categories: (1) *poisoning clients* that try to inject a specific malicious pattern; (2) *helper clients*, which have observed this specific malicious behavior before; their dataset contains correctly labeled data related to the malicious patterns; (3) other benign clients that have not seen the specific poisoning activity before. The helper clients are instrumental in sharing their knowledge through the global model to increase resilience against adversarial actions.

We compare the 'no poisoning' baseline with: (1) a data poisoning attack based on label flipping, where a number of malicious clients m change the label of malicious data to benign while computing the local model updates, and (2)

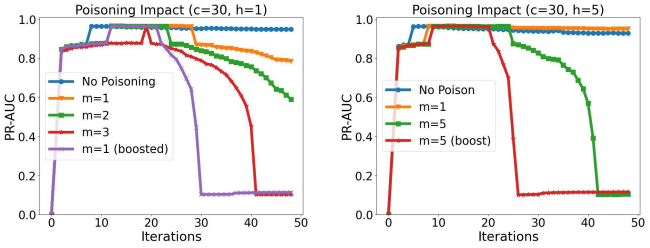

Fig. 5. Poisoning Attacks: Global model performance progress with various attack strategies, and no defense. We compare the 'no poisoning' baseline, the label flipping attack with m malicious clients, and a stronger 'weight boosting' attack. Poisoning starts at iteration 20. Increasing the number of helper clients h improves detection: $h = 1$ (left) and $h = 5$ (right).

state-of-the-art model poisoning attacks via boosting [19], [20], [64], which amplify or boost the weight updates of the poisoned clients to maximize the attack impact. The model poisoning attack is much stronger than data poisoning, as it allows the malicious clients to control the model updates. We study three poisoning scenarios with specific HTTP attacks: (1) Data exfiltration activity to the C2 server from Cosmic-Duke (part of our DEM dataset) [65]; (2) ThinkPHP exploit observed in Mirai [66]; and (3) a home-router attack with device update behavior from Gafgyt [67]. In these experiments, we consider a total of $c = 30$ clients, and vary the number m of malicious clients, as well as the number h of helper clients. We run the training for 48 iterations (one day of data, with 30-min long iterations), and start the poisoning activity at the 20th iteration.

In Figure 5, we show the impact of poisoning during the data exfiltration attack, when no defense is employed. We make the following observations: First, we note that the success of the label flipping attack increases with more poisoning clients m, as expected. Second, the global model is more resilient against poisoning attacks when more helper clients are contributing with locally seen attack patterns. With only one helper, a single malicious client is sufficient to decrease model accuracy substantially over time (left figure, with boosting); however, when five helpers participate in the global model, the attacker needs to compromise as many as five clients for an equivalent performance degradation (right figure, with boosting). Third, we note that model boosting attacks are significantly more effective than label flipping attacks, reaching a PR-AUC of 0.11 in both experimental settings (i.e., left and right figures) in only 10 and 6 iterations (after poisoning had started), respectively.

In Figure 6, we explore possible defenses against the model boosting attack; we showed in Figure 5 how this attack is particularly damaging, due to its ability to directly manipulate the model weights. We compare the 'no defense' baseline with: (1) the weight clipping technique, a state-of-the-art defense where the model updates are reduced to the clipping norm [19], [20], [64]; (2) our DTrust algorithm; and (3) a hybrid defense consisting of DTrust + weight clipping. We used clean training updates to determine the clipping bound parameter as 0.1, after observing that lower values significantly hinder the main task accuracy. The plot on the right ($m = 5, h = 5$)

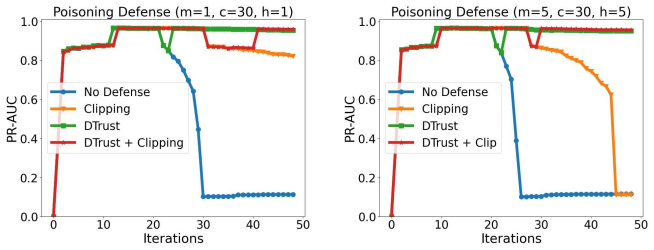

Fig. 6. Defenses: Global model performance progress when various defenses have been employed against the strongest attack from Figure 5 (weight boosting). DTrust successfully recovers from the poisoning attack, outperforming the 'no defense' baseline and the 'clipping' defense significantly.

TABLE IV

OVERVIEW OF THE POISONING EXPERIMENTS FOR THREE SCENARIOS AFTER ONE DAY OF TRAINING IN A 30-CLIENT SETTING. THE TRAINING FAMILY IS THE MAIN TASK THAT ALL CLIENTS TRAIN ON. WE RUN THE WEIGHT BOOSTING ATTACK WITH $m = 5$ MALICIOUS CLIENTS, AND $h = 5$ HELPERS. OUR DTRUST DEFENSE REACHES PR-AUC SCORES CLOSE TO THE CLEAN MODEL BY DETECTING THE POISONING ACTIVITY IN LESS THAN 4 ITERATIONS IN ALL THREE CASES

Training Family	Poisoning Behavior	No Defense	DTrust Defense	Clean Training
Mirai	Data Exfiltration	0.11	**0.93**	0.93
DEM	Home-Router Attack	0.08	**0.89**	0.92
DEM	ThinkPHP Exploit	0.06	**0.65**	0.70

demonstrates that the clipping technique alone is not sufficient to mitigate the impact of poisoning, as the PR-AUC still drops to 0.11 eventually during this strong attack. Our proposed defense, DTrust, starts to investigate the client updates after the global model PR-AUC deteriorates by more than 10%. DTrust identifies the malicious clients and removes their updates from the global model. The training continues with the benign clients, and the performance recovers over time. Eventually, DTrust reaches 0.93 PR-AUC, similar to the 'no poisoning' case. DTrust can be used as a standalone defense, or in combination with clipping (with similar results). CELEST is also effective in other poisoning scenarios with different HTTP attacks. Table IV summarizes the poisoning impact for the three attacks after one day of training, and presents the mitigation results. DTrust detects the poisoning activity in less than 4 iterations in each case. Furthermore, DTrust enables poisoned models to recover and reach close to the accuracy of the models trained on clean data.

In Table V, we compare the runtime performance of locally trained, centralized, and federated models over a full day of training, comprising 48 rounds in the federated setup on a single server. Centralized aggregation of multiple datasets leads to higher data preparation and training times compared to federated training. In federated learning experiments, the computational cost is primarily on the client side. The server performs only simple model weight aggregation, contributing minimally to the total training time. Local training and dataset preparation dominate the computation cost. We use the TensorFlow Federated framework, which simulates federated learning without modeling actual network latency. While communication overhead is a critical factor in real-world deployments, it is not captured in our simulations. Each federated learning

TABLE V

WE COMPARE THE RUNTIME PERFORMANCE OF LOCALLY TRAINED, CENTRALIZED, AND FEDERATED MODELS OVER A FULL DAY OF TRAINING, COMPRISING 48 ROUNDS IN THE FEDERATED SETUP. FEDERATED TRAINING PROVES MORE EFFICIENT THAN CENTRALIZED TRAINING. WHILE THE ADDITION OF THE DTRUST DEFENSE INCREASES COMPUTATIONAL OVERHEAD, ITS RUNTIME REMAINS COMPARABLE TO THAT OF THE CENTRALIZED MODEL

Training Experiment	Dataset Prep	Model Training	Total Time
Single Network (UNI-1)	21 min	10 min	32 min
Single Network (UNI-2)	17 min	9 min	27 min
Centralized	36 min	18 min	55 min
Federated	33 min	14 min	47 min
Federated with DTrust	33 min	18 min	51 min

round processes data collected over a fixed 30-minute window. Clients prepare and train on their local datasets during this interval, then send updates to the server at the end of the round. This design standardizes timing across clients and provides a consistent approximation of real-world computational demand. In multi-client settings, dataset preparation, which includes data loading and feature extraction, can be parallelized to reduce overall runtime. Regarding the defense mechanism against data poisoning, we observed minimal (8%) computational overhead overall. The defense involves clients performing inference on the global model using a fixed evaluation set, without additional training. Server-side cost increases only when a poisoning investigation is triggered, in which case the server runs inference for each client. Given the limited number of participating clients in our setup, this added cost remains negligible. However, this cost may be significant in other applications of federated learning scenarios involving a large number of devices. Overall, our implementation maintains computational efficiency while integrating defenses, and we believe the added overhead is minimal and scalable in small-to-medium federated deployments.

V. DISCUSSION AND EXTENSIONS

In this paper, we showed that global models that leverage coordinated detection across multiple participating organizations perform better at detecting malware threats compared to local models that only rely on limited local data. Federated learning is often able to expose new attacks, which are largely invisible to individual organizations. FL is particularly effective when used in conjunction with active learning, which progressively enhances the labeled dataset used in training with new malicious samples. When deployed on two university networks, CELEST was able to detect new malicious activity and identify evasive adversarial activity in real time.

Real-world Deployment. Active learning adds a major boost to the malware detection capabilities of a federated learning framework. Through the anomaly detection module integrated with the classifier that progressively improves its detection, CELEST can discover and learn the behavior of completely new attacks for which no labeled data is available. Our current framework uses a generic anomaly detector (Isolation Forest); however, in order to tap into its full potential,

a more sophisticated anomaly detector specifically designed for HTTP logs can be used. We showed in the evaluation that active learning can be effective even at small budgets of samples investigated and labeled by security analysts. We note that periodic training is necessary in order to learn changing patterns in background traffic, as well as to discover and incorporate new attack behaviors in the model. Another interesting extension is to support P2P architectures, where a group of autonomous peers jointly train a common model without using a trusted server. A decentralized FL approach avoids having a single point-of-failure and several designs have been proposed [68], [69], [70]. In addition, differential privacy techniques can be applied for the federated setting [71] to further protect client updates and mitigate the risk of information leakage.

HTTPS traffic. While CELEST is designed to detect HTTP malicious communication, an interesting challenge is encrypted malicious communication over HTTPS. Currently, HTTPS is not utilized by most malware due to the additional complexity in managing the certificates [72], but we expect that HTTPS usage will increase in the future. There are several avenues for handling malicious encrypted communication in CELEST. First, security proxies are commonly used in enterprise networks and CDNs to decrypt communication over HTTPS [73]. These proxies act as man-in-the-middle agents in TLS communication, making it possible to decrypt and inspect otherwise encrypted data in order to detect malware, data breaches, and policy violations. For such cases, CELEST will have access to all the fields available in the decrypted traffic, and can operate seamlessly. Second, if the traffic is not decrypted by proxies, CELEST would have access to limited information, but we verified that the system still performs well when only a subset of features is available. The overall system architecture is still applicable with a reduced feature set, and incorporating other types of security logs (e.g., DNS logs, connection logs, firewalls logs) would be beneficial to expand the feature list. We leave a detailed investigation of HTTPS communication detection as future work.

Other types of malware. While our current implementation is tailored to detecting HTTP-based malicious activity, the underlying framework is modular and can be adapted to analyze other types of network traffic or malware behaviors, such as those using different protocols or operating in different domains. Extending the framework would mainly involve adjusting the feature extraction process to accommodate the characteristics of new types of traffic or malware classes.

Resilience to Poisoning. Other studies have proposed to use a trust dataset [58] which resides on the server and is used to remove outlier updates or correct the model. In contrast, we employ a distributed trust bootstrapping approach where each client has its own trust dataset and actively participates in defense. We note that federated learning with privacy-preserving aggregation does not allow inspection of individual updates from the clients [74]. Our defense can be modified to simply revert to a previous model, but identification of malicious clients would not be possible in this case.

Another important class of threats in our setting is stealthy backdoor attacks where an adversary embeds targeted misclassification behavior triggered by rare or carefully crafted patterns, while maintaining high accuracy on clean data. Such attacks have been shown to be feasible in both federated learning [19], [75] and centralized malware classifiers [76], often bypassing standard defenses. In the case of DTrust, the defense relies on each client's trusted validation dataset to evaluate global model updates. If these datasets lack the specific trigger pattern, a backdoor may evade detection. We leave the experimental evaluation of stealthy backdoor attacks in federated malware-URL detection as an important direction for future work.

Resilience to Evasion. We performed a case study designed to emulate an adversarial evasion strategy, in which the adversary rotates both its C2 domain and IP infrastructure to evade detection. We showed that CELEST is still able to detect the malware attack when faced with this evasion method. CELEST's resilience can be attributed to two main factors: the large number of features used for training the federated model, and the participation of multiple clients. Still, a motivated attacker might be able to coordinate its behavior and generate traffic that looks normal across all participating clients to evade detection. We believe evasion is more difficult in a federated setting than in a local system trained within a single organization.

VI. Related Work

A. Malicious URL And Domain Detection

A large body of work has looked at URL-based detection [6], [7], [38], [39], [77], [78], [79] and domain-based detection [4], [5], [80], [81], [82] of malicious traffic. Some of these studies focus on one type of malware like phishing [78] or spam [77], [79] in social media contexts, while others attempt to capture a larger set of malware behaviors, e.g, Mamun et al. [7] group malicious URLs into categories like spam, phishing, malware, and defacement URLs using public datasets. Previous studies employ various techniques to detect malicious URL and domains. These include lexical features generated from bag-of-words representation [6], behavioral analysis [77], DNS data analysis [4], [82], webpage content analysis combined with some URL and domain properties [79], [80], and deep learning methods [7], [38], [39], [78], [81].

B. Malicious Http-Based Detection

Efforts on malware detection in this area have focused on HTTP-based detection using web-proxy logs [8], [10], [11], [12], [13], [28] and HTTP-based application fingerprinting [9], [29], [30]. Machine learning detection methods on HTTP logs attempt to detect malware activity within a single network by training supervised classification models locally on labeled data available in that network [8], [10], [12], [13]. In other approaches, Nelms et al. [11] use control protocol templates derived from labeled samples to detect new C2 domain names, while Bortolameotti et al. [9] use application fingerprinting techniques for clustering HTTP connections in order to detect anomalous traffic.

C. Federated Learning For Cyber Security

Federated learning has been proposed for enhancing cyber security in various settings including mobile phones [83], Internet-Of-Things [84], [85], and cloud ecosystems [86]. Khramtsova et al. [37] study federated learning approaches to malicious URL detection in order to show the benefit of sharing information about local detections. Zhao et al. [87] propose multi-task network anomaly detection using federated learning. Fereidooni et al. [16] propose federated learning to enable effective cyber-risk intelligence sharing for mobile devices. Several studies have looked at poisoning attacks against federated learning [19], [20], [21], [22], [23], [52], [75], [88], [89], [90]. Fang et al. [21] proposed a general framework of local model poisoning attacks, which can be applied to optimize the attacks for any given aggregation rule. Bagdasaryan et al. [19] have formulated adversarial poisoning as a two-task optimization problem that has high accuracy on both the main and backdoor tasks. Previously proposed defenses against poisoning in FL perform anomaly detection on client updates [53], [54], [55], are specific to backdoor attacks [56], [57], or leverage a trusted dataset at the server [58].

VII. Conclusion

In this paper, we present CELEST, a federated learning framework for collaborative threat detection. CELEST leverages a distributed machine learning architecture in which multiple participating organizations train a global model used for HTTP-based malware detection. Using a novel active learning component, CELEST progressively improves its detection capabilities. In addition, we propose DTrust, a new resilient algorithm aimed at defending against data and model poisoning attacks in distributed settings. We evaluate our system using a variety of malware datasets and demonstrate the power of knowledge transfer through the globally trained model, which enables individual organizations to detect attacks that were largely invisible locally. We deploy the model on two large university networks and show that CELEST is able to detect real-world malicious traffic (42 malicious URLs and 20 malicious domains). Overall, CELEST is a scalable and effective proactive threat detection solution that leverages collaboration across multiple networks to detect emerging cyber threats.

Acknowledgment

The authors would like to thank Afsah Anwar, Alastair Nottingham, Molly Buchanan, Mark Gardner, Jeffry Lang, and Jeffrey Collyer for their help throughout the project. This research was sponsored by contract number W911NF-18-C0019 with the U.S. Army Contracting Command-Aberdeen Proving Ground (ACC-APG) and the Defense Advanced Research Projects Agency (DARPA), W911NF-21-10322, and by the U.S. Army Combat Capabilities Development Command Army Research Laboratory under Cooperative Agreement Number W911NF-13-2-0045 (ARL Cyber Security CRA (Cyber Research Alliance)). The views contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the ACC-APG, DARPA, Combat Capabilities Development Command Army Research Laboratory or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation here on. This project was also funded by NSF under grant CNS-1717634.

References

[1] Symantec. (2017). *What You Need to Know About the WannaCry Ransomware*. [Online]. Available: https://symantec-blogs.broadcom.com/blogs/threat-intelligence/wannacry-ransomware-attack

[2] M. Antonakakis et al., "Understanding the mirai botnet," in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 1093–1110.

[3] L. Invernizzi et al., "Nazca: Detecting malware distribution in large-scale networks," in *Proc. 21st Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2014, pp. 688–703.

[4] L. Bilge, S. Sen, D. Balzarotti, E. Kirda, and C. Kruegel, "Exposure: A passive DNS analysis service to detect and report malicious domains," *ACM Trans. Inf. Syst. Secur.*, vol. 16, no. 4, pp. 1–28, Apr. 2014.

[5] R. D. Silva, M. Nabeel, C. Elvitigala, I. Khalil, T. Yu, and C. Keppitiyagama, "Compromised or attacker-owned: A large scale classification and study of hosting domains of malicious URLs," in *Proc. 30th USENIX Secur. Symp.*, 2021, pp. 3721–3738.

[6] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond blacklists: Learning to detect malicious Web sites from suspicious URLs," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jun. 2009, pp. 1245–1254.

[7] M. S. I. Mamun, M. A. Rathore, A. H. Lashkari, N. Stakhanova, and A. A. Ghorbani, "Detecting malicious URLs using lexical analysis," in *Proc. Int. Conf. Netw. Syst. Secur.*, 2016, pp. 467–482.

[8] K. Bartos, M. Sofka, and V. Franc, "Optimized invariant representation of network traffic for detecting unseen malware variants," in *Proc. 25th USENIX Secur. Symp.*, 2016, pp. 807–822.

[9] R. Bortolameotti et al., "DECANTeR: Detection of anomalous outbound HTTP traffic by passive application fingerprinting," in *Proc. 33rd Annu. Comput. Secur. Appl. Conf.*, Dec. 2017, pp. 373–386.

[10] X. Hu et al., "BAYWATCH: Robust beaconing detection to identify infected hosts in large-scale enterprise networks," in *Proc. 46th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Aug. 2016, pp. 479–490.

[11] T. Nelms, R. Perdisci, and M. Ahamad, "ExecScent: Mining for new C&C domains in live networks with adaptive control protocol templates," in *Proc. 22nd USENIX Secur. Symp.*, 2013, pp. 589–604.

[12] A. Oprea, Z. Li, R. Norris, and K. Bowers, "MADE: Security analytics for enterprise threat detection," in *Proc. 34th Annu. Comput. Secur. Appl. Conf. (ACSAC)*, Dec. 2018, pp. 124–136.

[13] A. Oprea, Z. Li, T.-F. Yen, S. H. Chin, and S. Alrwais, "Detection of early-stage enterprise infection by mining large-scale log data," in *Proc. 45th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Rio de Janeiro, Brazil, Jun. 2015, pp. 45–56.

[14] C. Johnson, L. Badger, D. Waltermire, J. Snyder, and C. Skorupka, *Guide to Cyber Threat Information Sharing*, Standard 800-150, National Institute of Standards and Technology, Oct. 2016.

[15] C. Wagner, A. Dulaunoy, G. Wagener, and A. Iklody, "MISP: The design and implementation of a collaborative threat intelligence sharing platform," in *Proc. ACM Workshop Inf. Sharing Collaborative Security (WISCS)*, 2016, pp. 49–56.

[16] H. Fereidooni, A. Dmitrienko, P. Rieger, M. Miettinen, A.-R. Sadeghi, and F. Madlener, "FedCRI: Federated mobile cyber-risk intelligence," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–16.

[17] V. Mavroeidis and S. Bromander, "Cyber threat intelligence model: An evaluation of taxonomies, sharing standards, and ontologies within cyber threat intelligence," in *Proc. Eur. Intell. Secur. Informat. Conf. (EISIC)*, Sep. 2017, pp. 91–98.

[18] W. Tounsi and H. Rais, "A survey on technical threat intelligence in the age of sophisticated cyber attacks," *Comput. Secur.*, vol. 72, pp. 212–233, Jan. 2018.

[19] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2938–2948.

[20] Z. Sun, P. Kairouz, A. Theertha Suresh, and H. Brendan McMahan, "Can you really backdoor federated learning?," 2019, *arXiv:1911.07963*.

[21] S. Lu, R. Li, X. Chen, and Y. Ma, "Defense against local model poisoning attacks to Byzantine-robust federated learning," *Frontiers Comput. Sci.*, vol. 16, no. 6, pp. 1605–1622, Dec. 2022.

[22] V. Shejwalkar and A. Houmansadr, "Manipulating the Byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1278–1295.

[23] H. Wang et al., "Attack of the tails: Yes, you really can backdoor federated learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 16070–16084.

[24] Azeria Labs. (2025). *Data Exfiltration Techniques*. [Online]. Available: https://azeria-labs.com/data-exfiltration/

[25] A. Remillano II and J. Urbanec. (May 2019). *New Mirai Variant Uses Multiple Exploits to Target Routers and Other Devices*. [Online]. Available: https://www.trendmicro.com/enus/research/19/e/new-mirai-variant-uses-multiple-exploits-to-target-routers-and-other-devices.html

[26] The Hacker News. (May 2020). *HTTP Status Codes Command This Malware How to Control Hacked Systems*. [Online]. Available: https://thehackernews.com/2020/05/malware-http-codes.html

[27] A. K. Sood, S. Zeadally, and R. J. Enbody, "An empirical study of HTTP-based financial botnets," *IEEE Trans. Dependable Secure Comput.*, vol. 13, no. 2, pp. 236–251, Mar. 2016.

[28] J. McGahagan, D. Bhansali, M. Gratian, and M. Cukier, "A comprehensive evaluation of HTTP header features for detecting malicious websites," in *Proc. 15th Eur. Dependable Comput. Conf. (EDCC)*, 2019, pp. 75–82.

[29] R. Bortolameotti et al., "HeadPrint: Detecting anomalous communications through header-based application fingerprinting," in *Proc. 35th Annu. ACM Symp. Appl. Comput.*, Mar. 2020, pp. 1696–1705.

[30] R. Perdisci, W. Lee, and N. Feamster, "Behavioral clustering of HTTP-based malware and signature generation using malicious network traces," in *Proc. 7th USENIX Symp. Networked Syst. Design Implement. (NSDI)*, 2010, p. 26.

[31] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, May 2010, pp. 305–316.

[32] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1, pp. 1–210, 2021.

[33] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, A. Singh and J.Zhu., Eds., Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.

[34] J. Bhatta, D. Shrestha, S. Nepal, S. Pandey, and S. Koirala, "Efficient estimation of nepali word representations in vector space," *J. Innov. Eng. Educ.*, vol. 3, no. 1, pp. 71–77, Mar. 2020.

[35] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. for Comput. Linguistics*, vol. 5, pp. 135–146, Dec. 2017.

[36] A. Blum, B. Wardman, T. Solorio, and G. Warner, "Lexical feature based phishing URL detection using online learning," in *Proc. 3rd ACM Workshop Artif. Intell. Secur.*, Oct. 2010, pp. 54–60.

[37] E. Khramtsova, C. Hammerschmidt, S. Lagraa, and R. State, "Federated learning for cyber security: SOC collaboration for malicious URL detection," in *Proc. IEEE 40th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Nov. 2020, pp. 1316–1321.

[38] J. Saxe and K. Berlin, "EXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys," 2017, *arXiv:1702.08568*.

[39] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," 2018, *arXiv:1802.03162*.

[40] D. Garcia Bernal, L. Giaretta, S. Girdzijauskas, and M. Sahlgren, "Federated Word2 Vec: Leveraging federated learning to encourage collaborative representation learning," 2021, *arXiv:2105.00831*.

[41] E. Lughofer, "Single-pass active learning with conflict and ignorance," *Evolving Syst.*, vol. 3, no. 4, pp. 251–271, Dec. 2012.

[42] D. D. Lewis and W. A. Gale, "A sequential algorithm for training text classifiers," in *Proc. Conf. Res. Develop. Inf. Retr.*, 1994, pp. 3–12.

[43] N. Roy and A. McCallum, "Toward optimal active learning through sampling estimation of error reduction," in *Proc. 18th Int. Conf. Mach. Learn. (ICML)*, 2001, pp. 441–448.

[44] D. A. Cohn, Z. Ghahramani, and M. I. Jordan, "Active learning with statistical models," *J. Artif. Intell. Res.*, vol. 4, pp. 129–145, Mar. 1996.

[45] J. W. Stokes, J. Platt, J. Kravis, and M. Shilman, "Aladin: Active learning of anomalies to detect intrusions," Microsoft Res., Redmond, WA, USA, Tech. Rep. MSR-TR-2008-24, 2008, Mar. 2008. [Online]. Available: https://www.microsoft.com/en-us/research/publication/aladin-active-learning-ofanomalies-to-detect-intrusions/

[46] B. Frenay and M. Verleysen, "Classification in the presence of label noise: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 25, no. 5, pp. 845–869, May 2014.

[47] J. M. Johnson and T. M. Khoshgoftaar, "A survey on classifying big data with label noise," *J. Data Inf. Qual.*, vol. 14, no. 4, pp. 1–48, Dec. 2022.

[48] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Jun. 2008, pp. 413–422.

[49] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," *ACM SIGMOD Rec.*, vol. 29, no. 2, pp. 93–104, Jun. 2000.

[50] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, Jul. 2001.

[51] R. J. G. B. Campello, D. Moulavi, A. Zimek, and J. Sander, "Hierarchical density estimates for data clustering, visualization, and outlier detection," *ACM Trans. Knowl. Discovery from Data*, vol. 10, no. 1, pp. 1–51, Jul. 2015.

[52] V. Tolpegin, S. Truex, M. E. Gürsoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Proc. 25th Eur. Symp. Res. Comput. Secur. (ESORICS)*, 2020, pp. 480–501.

[53] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 118–128.

[54] E. M. E. Mhamdi, R. Guerraoui, and S. Rouault, "The hidden vulnerability of distributed learning in byzantium," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 3521–3530.

[55] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2018, pp. 5650–5659.

[56] P. Rieger, T. Duc Nguyen, M. Miettinen, and A.-R. Sadeghi, "DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection," 2022, *arXiv:2201.00763*.

[57] T. D. Nguyen {et al.}, "FLAME: Taming backdoors in federated learning," in *Proc. 31st USENIX Security Symp. (USENIX Security)*, 2022, pp. 1415–1432.

[58] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. 28th Annu. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2021, pp. 1260–1277.

[59] V. Le Pochat, T. Van Goethem, S. Tajalizadehkhoob, M. Korczynski, and W. Joosen, "Tranco: A research-oriented top sites ranking hardened against manipulation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2019, pp. 1–15.

[60] O. Alrawi et al., "The circle of life: A large-scale study of the IoT malware lifecycle," in *Proc. 30th USENIX Secur. Symp. (USENIX Security)*. Berkeley, CA, USA: USENIX Association, Aug. 2021, pp. 3505–3522org/conference/usenixsecurity21/presentation/alrawi-circle.

[61] Keras Team. (2015). *Keras*. [Online]. Available: https://keras.io

[62] TensorFlow Federated Team. (2019). *TensorFlow Federated: Machine Learning on Decentralized Data*. [Online]. Available: https://www.tensorflow.org/federated

[63] R. Řehářek and P. Sojka, "Software framework for topic modelling with large corpora," in *Proc. LREC Workshop New Challenges NLP Frameworks*, May 2010, pp. 45–50.

[64] A. N. Bhagoji, S. Chakraborty, P. P. Mittal, and S. Calp, "Analyzing federated learning through an adversarial lens," in *Proc. 36th Int. Conf. Mach. Learn.*, May 2019, pp. 634–643.

[65] CYFIRMA Malware Research Team. (Jun. 2023). *CosmicDuke Malware Analysis Report*. [Online]. Available: https://www.cyfirma.com/outofband/cosmicduke-malware-analysis/

[66] A. Remillano. (Jan. 2019). *ThinkPHP Vulnerability Abused By Botnets Hakai and Yowai*. [Online]. Available: https://www.trendmicro.com/enus/research/19/a/thinkphp-vulnerability-abused-by-botnets-hakai-and-yowai.html

[67] SonicWall Capture Labs Threat Research Team. (Jul. 2019). *New Wave of Attacks Attempting to Exploit Huawei Home Routers*. [Online]. Available: https://www.sonicwall.com/blog/new-wave-of-attacks-attempting-to-exploit-huawei-home-routers

[68] L. Chou, Z. Liu, Z. Wang, and A. Shrivastava, "Efficient and less centralized federated learning," in *Machine Learning and Knowledge Discovery in Databases: Research Track*, vol. 12975. Cham, Switzerland: Springer, 2021.

[69] A. Gholami, N. Torkzaban, and J. S. Baras, "Trusted decentralized federated learning," in *Proc. IEEE Consum. Commun. Netw. Conf. (CCNC)*, Mar. 2022, pp. 1–6.

[70] T. Wink and Z. Nochta, "An approach for peer-to-peer federated learning," in *Proc. 51st Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. Workshops (DSN-W)*, Jun. 2021, pp. 150–157.

[71] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3454–3469, 2020.

[72] Sophos. (Apr. 2021). *Nearly Half of Malware Now Use TLS to Conceal Communications*. [Online]. Available: https://news.sophos.com/en-us/2021/04/21/nearly-half-of-malware-now-use-tls-to-conceal-communications

[73] E. Bursztein. (Jul. 2017). *Understanding the Prevalence of Web Traffic Interception*. [Online]. Available: https://elie.net/blog/security/understanding-the-prevalence-of-web-traffic-interception

[74] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1175–1191.

[75] C. Xie, K. Huang, P. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. 8th Int. Conf. Learn. Represent. (ICLR)*, 2020. [Online]. Available: https://openreview.net/forum?id=rkgyS0VFvr

[76] D. T. Nguyen, N. N. Tran, T. T. Johnson, and K. Leach, "PBP: Post-training backdoor purification for malware classifiers," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2025.

[77] C. Cao and J. Caverlee, "Detecting spam URLs in social media via behavioral analysis," in *Proc. 37th Eur. Conf. Inf. Retr. (ECIR)*, Aug. 2015, pp. 703–714.

[78] H. Yuan, Z. Yang, X. Chen, Y. Li, and W. Liu, "URL2Vec: URL modeling with character embeddings for fast and accurate phishing website detection," in *Proc. IEEE Int. Conf. Parallel Distrib. Process. Appl., Ubiquitous Comput. Commun., Big Data Cloud Comput., Social Comput. Netw., Sustain. Comput. Commun. (ISPA/IUCC/BDCloud/SocialCom/SustainCom)*, Dec. 2018, pp. 265–272.

[79] K. Thomas, C. Grier, J. Ma, V. Paxson, and D. Song, "Design and evaluation of a real-time URL spam filtering service," in *Proc. 32nd IEEE Symp. Secur. Privacy (S&P)*, Sep. 2011, pp. 447–462.

[80] D. Canali, M. Cova, G. Vigna, and C. Kruegel, "Prophiler: A fast filter for the large-scale detection of malicious web pages," in *Proc. 20th Int. Conf. World Wide Web*, Mar. 2011, pp. 197–206.

[81] Y. Li, K. Xiong, T. Chin, and C. Hu, "A machine learning framework for domain generation algorithm-based malware detection," *IEEE Access*, vol. 7, pp. 32765–32782, 2019.

[82] Y. Zhauniarovich, I. Khalil, T. Yu, and M. Dacier, "A survey on malicious domains detection through DNS data analysis," *ACM Comput. Surveys*, vol. 51, no. 4, pp. 1–36, Jul. 2019.

[83] R. Gálvez, V. Moonsamy, and C. Diaz, "Less is more: A privacy-respecting Android malware classifier using federated learning," *Privacy Enhancing Technol.*, vol. 2021, no. 4, pp. 96–116, Oct. 2021.

[84] T. D. Nguyen, S. Marchal, M. Miettinen, H. Fereidooni, N. Asokan, and A.-R. Sadeghi, "DÏoT: A federated self-learning anomaly detection system for IoT," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jul. 2019, pp. 756–767.

[85] G. Thamilarasu and W. Schneble, "Attack detection using federated learning in medical cyber-physical systems," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, Aug. 2019, pp. 1–8.

[86] J. Payne and A. Kundu, "Towards deep federated defenses against malware in cloud ecosystems," in *Proc. IEEE Conf. Technol. Homeland Secur. (TPS-ISA)*, Jan. 2019, pp. 92–100.

[87] Y. Zhao, J. Chen, D. Wu, J. Teng, and S. Yu, "Multi-task network anomaly detection using federated learning," in *Proc. 10th Int. Symp. Inf. Commun. Technol. (SoICT)*, Mar. 2019, pp. 273–279.

[88] P. Rieger, T. Krauß, M. Miettinen, A. Dmitrienko, and A.-R. Sadeghi, "CrowdGuard: Federated backdoor detection in federated learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–18.

[89] H. Fereidooni, A. Pegoraro, P. Rieger, A. Dmitrienko, and A.-R. Sadeghi, "FreqFed: A frequency analysis-based approach for mitigating poisoning attacks in federated learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–16.

[90] K. Kumari, P. Rieger, H. Fereidooni, M. Jadliwala, and A.-R. Sadeghi, "BayBFed: Bayesian backdoor defense for federated learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2023, pp. 737–754.